

■■

# AgentReliabilityLab

Interview Defense Pack

Agentic cyber alert triage · MITRE ATT&CK; · NIST CSF · CISA advisories  
LangGraph · Hybrid RAG · LangSmith Observability · RAGAS Eval · FastAPI · Docker

Sidharth Kriplani · [github.com/SidharthKriplani/agentreliabilitylab](https://github.com/SidharthKriplani/agentreliabilitylab)

| 6-node LangGraph | Agentic re-retrieval | MITRE ATT&CK v14 | NIST CSF 2.0     |
|------------------|----------------------|------------------|------------------|
| agent pipeline   | feedback loop        | CISA KEV corpus  | RAGAS eval layer |

## Contents

- 1. Project Overview & Problem Statement
- 2. Architecture Deep-Dive
- 3. The Agentic Loop — Why It Matters
- 4. Hybrid RAG Design
- 5. Deterministic Overrides
- 6. LangSmith Observability
- 7. RAGAS Evaluation Framework
- 8. Human-in-the-Loop (HITL)
- 9. Scaling Strategy
- 10. Interview Q&A; — 26 Questions

# 1. Project Overview & Problem Statement

---

Security Operations Centres (SOCs) at mid-to-large enterprises receive tens of thousands of raw alerts per day from EDR tools (CrowdStrike, SentinelOne), SIEM platforms (Splunk, Microsoft Sentinel), IDS/IPS systems (Snort, Suricata), and Syslog pipelines. The vast majority are noise. The 1% that are genuine threats — ransomware precursors, active lateral movement, credential dumping, C2 beaconing — require analyst attention within minutes.

Today, a Tier-1 SOC analyst manually reads each alert, looks up the relevant MITRE technique, checks the NIST control playbook, and decides: escalate to Tier-2, suppress, or monitor. This is slow (minutes per alert), inconsistent across analysts, and does not scale.

## **What AgentReliabilityLab does:**

- Accepts raw alert text in any format: EDR JSON, Syslog line, IDS rule trigger, SIEM event
- Retrieves relevant MITRE ATT&CK; techniques, NIST CSF 2.0 controls, and CISA advisories via hybrid RAG
- Uses a local LLM to map the alert to specific technique IDs, assign severity, extract IOCs, and recommend controls
- Routes to CRITICAL / HIGH / MEDIUM / LOW / BENIGN with policy citations
- Triggers human review automatically for CRITICAL and HIGH alerts
- Produces a tamper-evident audit log for every decision — no raw PII stored

## **What makes it an agent, not a pipeline:**

After the first retrieval + analysis pass, if the LLM's self-reported confidence is below threshold, the graph loops back to the retrieval node with a refined query that incorporates the partial analysis (technique hints, tactic names). This feedback loop — bounded at 2 attempts to prevent infinite loops — is what distinguishes an agent from a static pipeline. Most document processing pipelines are strictly linear. This one reasons about whether it knows enough to commit to a decision.

## 2. Architecture Deep-Dive

| Node        | Input                                                         | Output                                                                                     | Key design choice                                                                    |
|-------------|---------------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Intake      | raw_alert (str)                                               | alert_id, alert_source, alert_metadata                                                     | Heuristic source detection (EDR JSON / Syslog / IDS); deterministic benign pre-check |
| Retrieve    | raw_alert, metadata, retrieval_attempts                       | retrieved_chunks, retrieval_query                                                          | Hybrid BM25 + dense; query enriched on re-retrieval with partial analysis context    |
| Analyze     | alert text, chunks, metadata                                  | TriageAnalysis Pydantic model (technique_ids, severity, confidence, requires_more_context) | 3-pass JSON recovery; cache SHA-256(text+attempt); benign fast-path skips LLM        |
| [Loop edge] | analysis.requires_more_context AND attempts OR Max Confidence | Retrieve (refined)                                                                         | Conditional edge — the agentic feedback loop                                         |
| Confidence  | analysis dict                                                 | confidence_score, confidence_band (H/M/L)                                                  | Heuristic adjustment: +0.05 per technique ID, -0.10 for UNKNOWN severity             |
| Route       | analysis, confidence, triage_decision                         | triage_decision, reason_codes, human_review_required                                       | Technique-level hard overrides (T1486→CRITICAL); low confidence upgrades LOW→MEDIUM  |
| Audit       | full AlertState                                               | audit_id, JSON file                                                                        | Chunk IDs only (not text); LangSmith run_id pointer; token economics logged          |

### State schema (AlertState TypedDict):

```
raw_alert · alert_id · alert_source · alert_metadata · retrieval_query ·
retrieved_chunks · retrieval_attempts · analysis (TriageAnalysis) · confidence_score ·
confidence_band · triage_decision · reason_codes · human_review_required ·
human_override · audit_id · token_log · langsmith_run_id · pipeline_version · error ·
error_node
```

### 3. The Agentic Loop — Why It Matters

---

The defining feature of AgentReliabilityLab vs. a simple RAG pipeline is the conditional edge from the analyze node back to the retrieve node.

#### How it works:

- First retrieval pass: query is built from raw alert text + extracted metadata (IPs, CVEs, process names)
- LLM analyzes alert + retrieved chunks, produces TriageAnalysis with a `requires_more_context` flag
- If `requires_more_context=True` AND `retrieval_attempts < MAX_RETRIEVAL_ATTEMPTS` (default: 2): the graph loops back to retrieve
- Second retrieval pass: query is enriched with `technique_ids` and `tactics` from the partial analysis — much more specific
- LLM re-analyzes with richer context, sets `requires_more_context=False`, pipeline proceeds to confidence → route → audit

#### The conditional edge code:

```
def _should_reretrieve(state): analysis = state.get("analysis", {}) attempts = state.get("retrieval_attempts", 0) if analysis.get("requires_more_context") and attempts < MAX_RETRIEVAL_ATTEMPTS: return "retrieve" # loop back return "confidence" # proceed
```

This is wired as a LangGraph `conditional_edge` — the graph itself encodes the reasoning loop, not application-level code. This makes it debuggable in LangSmith, checkpointable for HITL, and testable in isolation.

#### Why bounded at 2 attempts?

Unbounded loops are a reliability anti-pattern in agentic systems. After 2 retrieval passes, the quality of additional context diminishes rapidly while latency grows linearly. The `MAX_RETRIEVAL_ATTEMPTS` config key makes this tunable per deployment without code changes.

## 4. Hybrid RAG Design

The knowledge base covers three corpora: MITRE ATT&CK; v14 technique descriptions, NIST CSF 2.0 core controls, and CISA Known Exploited Vulnerability advisories. All chunked at 400 characters with 80-character overlap and indexed at startup.

| Layer            | Tool                                 | Why                                                                         |
|------------------|--------------------------------------|-----------------------------------------------------------------------------|
| Dense retrieval  | ChromaDB + all-MiniLM-L6-v2          | Semantic similarity — catches paraphrased technique descriptions            |
| Sparse retrieval | BM25 (rank-bm25)                     | Exact keyword match — catches CVE IDs, technique IDs, specific tool names   |
| Reranking        | cross-encoder/ms-marco-MiniLM-L-6-v2 | Rescores merged candidate set; eliminates off-topic chunks that scored well |
| Deduplication    | chunk_id keyed dict                  | Same chunk from BM25 + dense → keep highest score, not duplicated           |

BM25 is particularly important for MITRE ATT&CK; data because technique IDs (T1059.001), CVE identifiers (CVE-2021-44228), and tool names (Mimikatz, ProcDump) are exact-match keywords that semantic embeddings may not rank highly. The cross-encoder then rescores the merged set end-to-end, capturing relevance that neither sparse nor dense alone achieves.

## 5. Deterministic Overrides

Two categories of deterministic logic bypass or override the LLM, ensuring the system is reliable even when the model makes mistakes:

### Category A — Technique-level hard overrides (route node):

| Technique                         | Override   | Rationale                                         |
|-----------------------------------|------------|---------------------------------------------------|
| T1486 — Data Encrypted for Impact | → CRITICAL | Ransomware payload in progress                    |
| T1490 — Inhibit System Recovery   | → CRITICAL | Ransomware precursor (vssadmin shadow deletion)   |
| T1485 — Data Destruction          | → CRITICAL | Irreversible damage in progress                   |
| T1055 — Process Injection         | → HIGH     | Active code injection — high confidence malicious |
| T1003 — OS Credential Dumping     | → HIGH     | LSASS/Mimikatz activity                           |
| T1021 — Remote Services           | → HIGH     | Lateral movement                                  |
| T1071 — C2 Protocol               | → HIGH     | Active C2 channel                                 |
| T1190 — Exploit Public-Facing App | → HIGH     | Active exploitation attempt                       |

### Category B — Deterministic benign fast-path (intake node):

If the raw alert text matches a known-safe phrase pattern (e.g. 'scheduled' + 'deploy', 'nessus' + 'scan'), the intake node sets `alert_metadata['deterministic_benign']=True`. The analyze node detects this flag and returns a pre-built BENIGN TriageAnalysis with 0.95 confidence — no LLM call, no retrieval. Observed result: 0.4s latency, 0 tokens.

This mirrors LendFlow's deterministic FOIR recomputation: never use a probabilistic model for what you can compute with certainty. Both projects use the same principle — LLM for judgment calls, deterministic code for calculable facts.

## 6. LangSmith Observability

---

LangSmith is enabled via `LANGCHAIN_TRACING_V2=true` in `.env`. When active, every pipeline run produces a trace in the LangSmith dashboard showing:

- Per-node latency breakdown (intake / retrieve / analyze / confidence / route / audit)
- Token cost per LLM call (prompt tokens, completion tokens)
- Number of retrieval attempts and the query used on each pass
- The full prompt sent to the LLM and the raw response received
- The LangSmith `run_id` is stored in the audit log — every audit record is linked to its trace

### Why this matters in interviews:

Production agentic systems fail in ways that are invisible without observability: the LLM decides to loop twice on a simple alert (cost spike), retrieval returns off-topic chunks (faithfulness drops), a node times out silently. Print statements don't scale to 10,000 alerts/day. LangSmith is the difference between debugging in production and having a structured trace to replay.

The design is opt-in: `LANGCHAIN_TRACING_V2=false` (default) means the pipeline runs identically with zero external dependencies. One env variable flips full observability on.

## 7. RAGAS Evaluation Framework

Most RAG systems are evaluated only on their final output accuracy. RAGAS measures retrieval quality independently — separating two failure modes:

| RAGAS Metric      | What it measures                                  | Failure it catches                                         |
|-------------------|---------------------------------------------------|------------------------------------------------------------|
| faithfulness      | Does the answer contradict the retrieved context? | Context hallucinating technique IDs not in chunks          |
| context_recall    | Did retrieval fetch all the relevant chunks?      | Missing MITRE technique for the attack pattern             |
| context_precision | Are retrieved chunks actually useful?             | Low-quality chunks polluting the context window            |
| answer_relevancy  | Does the answer address the question?             | LLM producing generic security advice vs. specific mapping |

The full eval harness (scripts/run\_eval.py) measures triage accuracy, severity accuracy, MITRE technique recall, and latency across 20 synthetic alerts. RAGAS (eval/ragas\_eval.py) then measures retrieval quality separately — because you can have high triage accuracy while retrieval is actually broken (the LLM compensates with parametric knowledge).

## 8. Human-in-the-Loop (HITL)

---

The production graph (`build_graph()`) is compiled with `interrupt_before=['audit']`. When a CRITICAL or HIGH alert is routed, the graph pauses before writing the audit log, allowing an analyst to review the LLM's triage decision and override if needed.

### Two graph variants:

- `graph = build_graph()` — compiled WITH `interrupt_before=['audit']`. Used by the FastAPI `/triage` endpoint. Pauses for human review on CRITICAL/HIGH.
- `graph_no_interrupt = build_graph_no_interrupt()` — audit always runs. Used by `demo.py`, `run_eval.py`, and all tests. Audit ID always populated.
- `run_pipeline()` convenience wrapper always uses `graph_no_interrupt` — prevents the common bug where `audit_id` shows '—' in batch runs.

### Resume flow (API):

POST `/human-review/override` with `{thread_id, alert_id, override_decision, analyst_notes}`. In production this calls `graph.update_state()` to inject the analyst override, then `graph.stream()` to resume from the interrupted checkpoint and complete the audit.

## 9. Scaling Strategy

| Bottleneck        | Current                           | Production fix                                                              |
|-------------------|-----------------------------------|-----------------------------------------------------------------------------|
| LLM throughput    | Local LM Studio (1 req at a time) | Swap LLM_BASE_URL to OpenAI / Azure OpenAI — 3 env vars, 0 code changes     |
| Retrieval latency | ChromaDB in-process               | ChromaDB server mode or Qdrant cluster; retriever is behind an interface    |
| Cache             | In-memory dict (process-local)    | Redis cluster — REDIS_URL env var, zero code changes                        |
| HITL state        | MemorySaver (in-memory)           | SqliteSaver or PostgresSaver for persistence across restarts                |
| Alert ingestion   | POST /triage (sync)               | Kafka topic → worker pool → async graph.astream()                           |
| Audit storage     | Local JSON files                  | S3 + DynamoDB or PostgreSQL; audit_node just needs a writer interface       |
| Eval CI           | pytest + mock LLM                 | eval-on-PR via GitHub Actions — already wired in .github/workflows/eval.yml |

## 10. Interview Q&A; — 26 Questions

---

### Q1. What is AgentReliabilityLab and what problem does it solve?

It's a LangGraph agent that triages raw security alerts against a hybrid RAG knowledge base of MITRE ATT&CK, NIST CSF 2.0, and CISA advisories. The problem: SOC analysts manually read thousands of alerts per day to find the handful that are genuine threats. ARL automates Tier-1 triage — classifying severity, mapping to MITRE techniques, recommending controls, and flagging for human review — in seconds per alert.

### Q2. What makes this an agent vs. a pipeline?

The conditional edge from analyze → retrieve. If the LLM's self-reported confidence is below threshold after the first retrieval pass, the graph loops back with a refined query built from the partial analysis. A pipeline is static and linear. An agent reasons about whether it has enough information to commit to a decision and can request more context. That feedback loop is the agent.

### Q3. Why LangGraph instead of a simple Python loop?

LangGraph gives you: (1) stateful checkpointing — the graph can be paused mid-run for HITL and resumed after an analyst override; (2) conditional edges as first-class graph topology — the re-retrieval loop is declared in the graph, not buried in application code; (3) LangSmith integration — every node is traceable with per-call latency and token cost. A Python loop gives you none of that.

### Q4. Explain the hybrid RAG retrieval strategy.

BM25 (sparse) handles exact keyword matches — CVE IDs, MITRE technique IDs, tool names like Mimikatz. ChromaDB with sentence-transformers (dense) handles semantic similarity — catching paraphrased descriptions of attack patterns. The cross-encoder reranker rescores the merged candidate set end-to-end, eliminating chunks that scored well on one signal but aren't actually relevant. This is the same BM25 + dense + reranker stack used in LendFlow — proven to outperform either retrieval method alone.

### Q5. Why use BM25 specifically for security data?

Security documents contain exact identifiers that embedding models struggle with: 'T1059.001', 'CVE-2021-44228', 'vssadmin.exe delete shadows /all /quiet'. These are low-frequency tokens — the embedding model may not have a strong representation for them. BM25 is exact-match by design. When an alert contains 'CVE-2021-44228', BM25 will rank the Log4Shell advisory at the top regardless of what the embeddings say.

### Q6. How does the confidence scoring work?

The LLM self-reports a confidence float (0-1) in the TriageAnalysis. The confidence node applies heuristic adjustments: +0.05 per MITRE technique ID present (max +0.15), +0.05 per NIST control recommended (max +0.10), -0.10 if severity is UNKNOWN, -0.05 if no technique IDs. The adjusted score maps to HIGH (≥0.80), MEDIUM (≥0.55), LOW (<0.55). LOW confidence triggers human review regardless of severity.

### Q7. Why are some techniques hard-coded as CRITICAL overrides?

T1486 (ransomware encryption) and T1490 (shadow copy deletion) are unambiguous ransomware precursors. Waiting for the LLM to correctly classify them introduces reliability risk — a 4B model might assign HIGH instead of CRITICAL. The deterministic override ensures these are never silently downgraded. Same principle as LendFlow's FOIR recomputation: never trust a probabilistic model for what you can compute with certainty.

### Q8. What is the deterministic benign fast-path?

If the intake node detects known-safe phrases — ('scheduled' + 'deploy'), ('nessus' + 'scan'), ('ci/cd' + 'deploy') — it sets alert\_metadata['deterministic\_benign']=True. The analyze node detects this flag and returns a pre-built BENIGN TriageAnalysis at 0.95 confidence without calling the LLM. Observed: 0.4s latency, 0 tokens. This prevents the model from hallucinating threat techniques onto clearly benign activity.

#### Q9. How does the HITL flow work?

The production graph is compiled with `interrupt_before=['audit']`. For CRITICAL/HIGH alerts, the graph pauses before writing the audit log. The FastAPI `/human-review/override` endpoint accepts the analyst's decision. In production, this calls `graph.update_state()` to inject the override into the checkpoint, then `graph.stream()` to resume. The audit log captures both the LLM's original decision and the analyst override with a reason field.

#### Q10. Why two graph variants (graph vs. graph\_no\_interrupt)?

The `interrupt_before=['audit']` compilation means the audit node never runs in batch contexts — `demo.py`, `run_eval.py`, `tests` — because those don't use the HITL resume flow. Every alert would show `audit_id='—'`. The fix: compile a second graph without interrupt for non-HITL contexts. `run_pipeline()` always uses `graph_no_interrupt`. The HITL graph is only used by the FastAPI endpoint where the resume flow is wired.

#### Q11. How does LangSmith fit in?

When `LANGCHAIN_TRACING_V2=true`, LangGraph automatically sends a trace to LangSmith for every graph execution. Each node appears as a span with input state, output state, latency, and token cost. The `langsmith_run_id` is captured and stored in the audit log — every audit record links back to its full LangSmith trace. This is the observability story: not print statements, but a queryable trace for every production run.

#### Q12. What does RAGAS measure that your custom eval doesn't?

The custom eval (`run_eval.py`) measures triage accuracy — did the final CRITICAL/HIGH/MEDIUM/LOW/BENIGN decision match ground truth? RAGAS measures retrieval quality independently: faithfulness (does the LLM's answer contradict the chunks?), `context_recall` (were the right chunks retrieved?), `context_precision` (are retrieved chunks useful or noisy?). You can have 90% triage accuracy while retrieval is broken — the LLM compensates with parametric knowledge. RAGAS catches that.

#### Q13. What are the known limitations?

Model quality: Gemma 3 4B misclassifies some borderline alerts — the same constraint as LendFlow's 68.8% field extraction F1. Critical routing fields (ransomware techniques, C2 patterns) are captured reliably via deterministic overrides. Latency: 15-40s per alert on local hardware — acceptable for Tier-1 async triage, not for real-time blocking. Fix: GPT-4o-mini via one env var change. Knowledge base: the `threat_docs` are curated excerpts, not the full MITRE corpus. Production would use the full STIX/TAXII feed.

#### Q14. How would you scale this to 10,000 alerts per day?

Replace LM Studio with OpenAI API (3 env vars). Use Kafka for alert ingestion — workers call `graph.astream()` asynchronously. Replace MemorySaver with PostgresSaver for HITL state persistence. Replace Redis-optional cache with Redis cluster. ChromaDB → Qdrant in server mode for concurrent reads. Audit logs → S3 + DynamoDB. The pipeline code requires zero changes — only the infrastructure wiring changes.

#### Q15. How is this different from Splunk SOAR or Palo Alto XSOAR?

SOAR platforms are rule-based playbook executors — a human writes 'if alert.type == ransomware then isolate host'. They don't understand unstructured alert text, they don't do RAG retrieval, and they don't map to MITRE techniques automatically. ARL uses an LLM + knowledge base to understand arbitrary alert formats and reason about threat intel. The tradeoff: SOAR is deterministic and auditable; ARL is flexible but probabilistic. Production would use ARL for Tier-1 classification feeding a SOAR playbook for the response actions.

#### Q16. Why sentence-transformers/all-MiniLM-L6-v2 for embeddings?

It's small (22M parameters), fast (<10ms per embedding on CPU), and performs well on asymmetric search (short query vs. longer document chunk). It runs fully locally — no API calls, no data egress. For a production deployment where the knowledge base is updated weekly, you'd want a larger model (e7-large, BGE-large) or a fine-tuned security-domain model. The embedding model is behind a config variable — swap in one line.

### Q17. How does the cache work?

SHA-256(alert\_text + retrieval\_attempt\_number) → Redis (if REDIS\_URL is set) or in-memory dict (fallback). The key includes the attempt number because attempt 0 and attempt 1 have different user prompts (attempt 1 includes partial analysis context). TTL: 24 hours. The cache stores the validated TriageAnalysis dict — identical alerts skip the LLM entirely on re-submission. This is critical for eval runs where the same alert may be processed multiple times.

### Q18. What's in the audit log?

audit\_id, alert\_id, alert\_source, timestamp, pipeline\_version, triage\_decision, reason\_codes, human\_review\_required, human\_override, technique\_ids, tactics, severity, severity\_rationale, recommended\_controls, ioc\_count (not the IOCs themselves), confidence\_score, confidence\_band, retrieval\_attempts, retrieved\_chunk\_ids (not the chunk text), token\_log, langsmith\_run\_id, error, error\_node. No raw alert text is stored — only metadata. No full chunk text — only chunk IDs that can be looked up.

### Q19. Why store chunk IDs instead of chunk text in the audit log?

The retrieved chunks can be large (400 chars × 8 chunks = 3.2KB per record). At 10,000 alerts/day that's 32MB/day in chunk text alone. More importantly, chunk text is reproducible — given the chunk\_id and the indexed knowledge base, you can always retrieve the exact text. Storing IDs instead of text is the same pattern as LendFlow's pii\_map\_id — the audit log contains a pointer, not the data.

### Q20. How do you handle LLM JSON parse failures?

3-pass recovery: (1) direct json.loads(); (2) strip markdown code fences with regex; (3) regex search for outermost {...} block. If all three fail, the node retries up to MAX\_RETRIES times, appending a correction message to the conversation. If still empty after retries, returns an error state — the graph propagates the error to audit. This is identical to LendFlow's extract node — a proven pattern for local models that leak markdown.

### Q21. How does error handling work across nodes?

Every node checks state.get('error') at entry and returns {} immediately if set. Errors are accumulated with error and error\_node fields so the audit log captures exactly which node failed. The graph always reaches the audit node regardless of errors — you get a complete audit record even for failed runs.

### Q22. What's the test strategy?

test\_intake.py: unit tests for source detection, metadata extraction, CVE/IP regex, error handling — no LLM dependency. test\_routing.py: unit tests for severity-to-decision mapping, technique-level hard overrides, confidence band logic — no LLM. test\_e2e.py: end-to-end with mocked OpenAI client — tests full graph traversal, state field population, token log accumulation, audit ID generation. GitHub Actions runs intake + routing unit tests on every PR, plus mocked E2E tests.

### Q23. Why FastAPI instead of Flask?

Async-native, Pydantic request/response validation built-in, automatic OpenAPI docs at /docs, type-annotated routes. FastAPI + LangGraph's astream() gives a natural async story for production. More practically: FastAPI is what every backend engineer interviewing you is using in 2025.

### Q24. How does this compare to LendFlow in terms of skill demonstration?

LendFlow demonstrates: document pipeline, PII redaction, structured extraction, deterministic rule engine, hybrid RAG for policy retrieval, HITL, audit logging. AgentReliabilityLab demonstrates: agentic feedback loop (different problem class), observability-first design with LangSmith, RAGAS retrieval evaluation, security domain knowledge (MITRE/NIST/CISA), deterministic technique overrides. The two projects together cover Applied LLM Systems + AI Infrastructure + domain-specific knowledge bases across two completely different verticals.

**Q25. What would you add to make this production-ready?**

1. Full MITRE ATT&CK; STIX/TAXII feed ingestion (not curated excerpts). 2. Real-time CISA KEV feed polling via scheduler. 3. Async alert ingestion via Kafka topic. 4. Structured LLM output via function calling / tool use (eliminates JSON recovery logic). 5. A feedback loop where analyst overrides are logged and used to fine-tune the embedding model or few-shot prompts over time. 6. Rate limiting, auth, and tenant isolation on the FastAPI layer.

**Q26. If you had to pick one thing that would impress a senior engineer, what is it?**

The two-graph compilation pattern (graph vs. graph\_no\_interrupt). It's a subtle but real production bug — compiling with `interrupt_before=['audit']` means the audit node never runs in batch/demo/eval contexts, so `audit_id` is always null. Discovering that, understanding why it happens (LangGraph's interrupt semantics), and fixing it cleanly by exposing two named graph variants rather than a flag or try/except — that's the kind of thing that shows you've actually run this in production conditions, not just written code that looks good on GitHub.

End of Defense Pack  
[github.com/SidharthKriplani/agentreliabilitylab](https://github.com/SidharthKriplani/agentreliabilitylab)